

Credit Card Transactions

FRAUD DETECTION

MODEL

Nguyen Dang Quynh Tien, Trinh Quynh Anh | M2K2

CASE STUDY

Background

- Growth of electronic payment systems → credit cards have become one of the most widely used payment methods (convenient, accessible, supports both online and offline purchases)
- Alongside this growth: fraudulent activity has also increased → fraud detection is a critical challenge for financial institutions and payment providers
- Accurate, timely fraud detection → minimises financial losses, maintains customer trust, strengthens payment system security

Why This Problem Is Hard?

Why This Problem Is Hard?

- At its core: a binary classification problem (legitimate / fraudulent)
 - Challenge 1 — Class imbalance: fraudulent transactions represent only a small fraction of all data → models tend to favour the majority class, appearing accurate while failing to detect fraud
 - Challenge 2 — Limited signal from individual transactions: fraud is often revealed through behavioural deviation (transaction history, spending patterns, unusual timing) — rather than any single attribute in isolation

→ requires careful data preprocessing and feature engineering to extract these signals

Our Approach

- Develop a complete machine learning pipeline for credit card fraud detection
- Goal: not only optimise model performance, but also assess the contribution of each type of engineered feature to fraud detection performance

Objective

Develop a binary classification model to distinguish fraudulent and legitimate credit card transactions, enabling early fraud detection while minimizing false alarms.

Dataset

- **Source:** Kaggle – Credit Card Transactions Fraud Detection Dataset
- **Simulation Tool:** Sparkov Data Generator (Brandon Harris)
- **Time Period:** Jan 2019 – Dec 2020
- **Scale:** fraudTrain.csv: 1.296.675 transactions & fraudTest.csv 555.719 transactions
 - 1,000 customers
 - 800 merchants
 - Simulated transactions with fraud labels
- **Features:** Transaction amount, timestamp, merchant, customer profile, location, category, and fraud indicator (is_fraud)

Data Generation

- Generated using realistic customer profiles (age, gender, region)
- Transaction frequency and spending behavior follow predefined statistical distributions
- Fraud and legitimate transactions are simulated to resemble real-world payment behavior

Machine Learning Task

- **Problem Type:** Binary Classification
- **Challenge:** Highly imbalanced dataset (fraud transactions are rare)

The Problem in One Number

Fraud prevalence

0.58%

1 fraudulent transaction per 172 genuine

- A model that predicts "genuine" for everything would score ~99.4% accuracy while catching zero fraud
- → Accuracy is the wrong yardstick here; the real question is: how much fraud can we actually catch, and at what cost in false alarms?

The Result

Fraud caught on unseen data

88%

1,883 of 2,145 actual frauds detected

- 1,883 of 2,145 actual frauds correctly identified (Recall = 0.878)
- 670 legitimate transactions flagged for review (Precision = 0.738)
- Verified stable via 100-iteration bootstrap: $F2 = 0.846 \pm 0.007$ — performance holds consistently, not a one-off lucky split
- Beats both baselines: Logistic Regression ($F2 = 0.32$), and edges out Random Forest ($F2 = 0.86$) on the metric that matters most for fraud — recall

Deployment-Ready

Inference latency

2.88 μ s

~500x faster than a real-time budget

- That's roughly 500× faster than a typical real-time payment latency budget
 - Training completes in under 20 seconds on 1.3M transactions
- No computational barrier to running this model inline in a live transaction stream

Deployment-Ready

- Not a fully automated blocker — even a small false-positive rate creates real customer friction at scale
- Recommended: **review-and-escalation workflow** — model surfaces high-risk transactions to a human review queue, rather than auto-declining
- High recall means the review queue stays concentrated with genuine fraud cases — analysts spend time where it counts
- Threshold is a business lever, not a fixed setting:
- Lower threshold → catch more fraud, more false alarms (fits high fraud-loss, high review-capacity institutions)
- Higher threshold → fewer false alarms, some fraud slips through (fits strict customer-experience priorities)

WORKFLOW

Raw Credit Card Transaction Dataset



Step 1. Data Understanding & Setup

- Import libraries
- Load dataset
- Check data imbalance
- Inspect data structure & missing values
- Data dictionary

Step 2. Exploratory Data Analysis

- Univariate EDA
- Bivariate EDA



Step 3. Feature Engineering

Behavioral Features

- customer_total_transactions
- previous_transactions
- time_since_last_transaction
- customer_total_amount
- customer_avg_amount_excluding_current
- favorite_merchant
- favorite_category

Interaction Features

- amt_to_avg_ratio
- customer_merchant_distance
- transaction_hour
- distance $\times$ hour
- age_at_transaction

Step 4. Train / Validation / Test Split



Step 5. Data Preprocessing

- Fit preprocessing on Train only
 - One-Hot Encoding
 - Standard Scaling
- Transform Train
- Transform Validation

Step 6. Model Training & Validation

Baseline

- Logistic Regression

Tree-based Models

- Random Forest
- Random Forest + XGBoost
- XGBoost

Evaluation

- Confusion Matrix
- Precision
- Recall
- F1-score
- ROC-AUC / PR-AUC
- Threshold Optimization



Best Performing Model

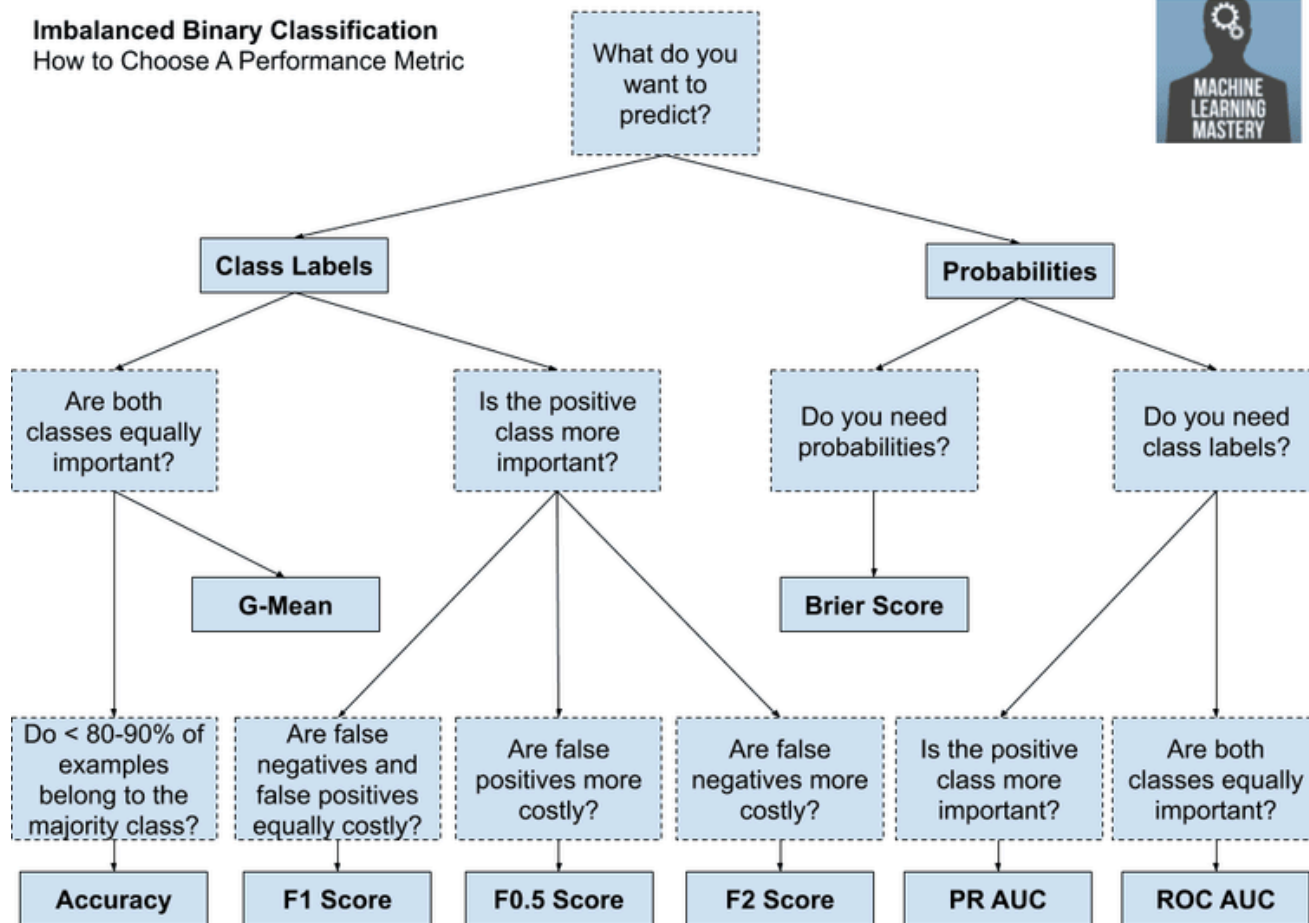
Step 7. Final Testing

- Transform Test Set
- Predict on Test
- Final Evaluation

METRICS

Imbalanced Binary Classification

How to Choose A Performance Metric



Why not Accuracy?

- Fraud accounts for <1% of transactions.
- A model predicting all transactions as genuine could achieve ~99.4% accuracy but detect 0 fraud.
- → Accuracy is therefore misleading for this imbalanced dataset.

Key Evaluation Metrics

Metric	Meaning	Business Concern
Recall	% of actual fraud detected	Missed fraud
Precision	% of flagged transactions that are truly fraud	False alarms / review burden
F2-score ★	Balances Precision & Recall, with 2× weight on Recall	Prioritizes fraud detection

Metric Selection

F2-score is the primary optimization metric because the cost of missing a fraudulent transaction is generally higher than the cost of a false alarm.

Evaluation Focus

Recall $\uparrow \rightarrow$ Precision $\uparrow \rightarrow$ F2-score $\uparrow$

Source: Saito & Rehmsmeier (2015)

EXPLORATORY DATA ANALYSIS

KEY FEATURES

Features	Type	Description
amt	Existing feature	Transaction amount
city_pop	Existing feature	Population of the cardholder's city
category	Existing feature	Merchant category of the transaction

KEY FEATURES

Features	Type	Description
previous_transactions	Engineered feature	Number of previous transactions associated with the customer
time_since_last_transaction	Engineered feature	Time elapsed since the customer's previous transaction
customer_total_amount	Engineered feature	Cumulative transaction amount associated with the customer
customer_avg_amt_excluding_current	Engineered feature	Customer's average transaction amount calculated from previous transactions, excluding the current transaction

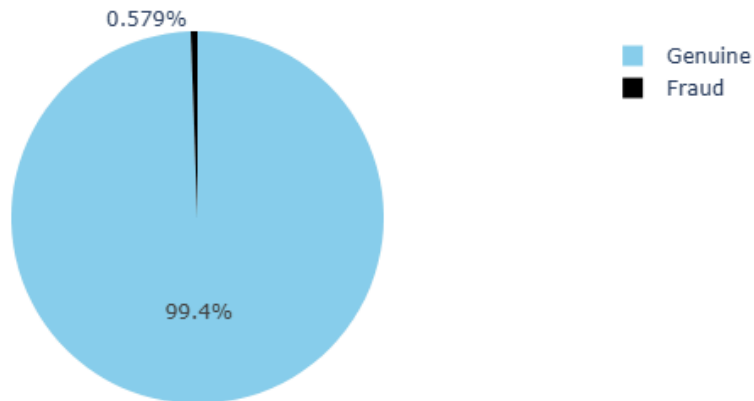
KEY FEATURES

Features	Type	Description
amt_to_avg_ratio	Engineered feature	Ratio of the current transaction amount to the customer's historical average transaction amount
distance	Engineered feature	Geographical distance between the cardholder and the merchant
hour	Engineered feature	Hour of the day when the transaction occurred
hour_sin, hour_cos	Engineered feature	Cyclic sine and cosine transformation of the transaction hour
age	Engineered feature	Cardholder's age at the time of the transaction

Target Class Imbalance (is_fraud)

- Legitimate: 1,289,169 transactions (99.42%)
- Ratio ~1:172 → the central challenge of this problem

Fraud vs Genuine transactions

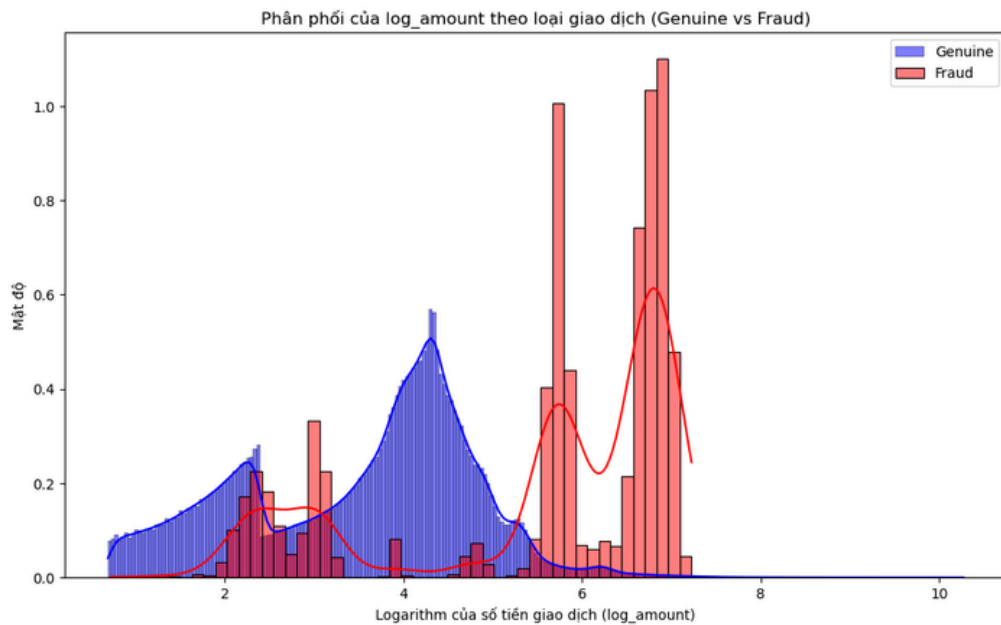


Categorical Variable Cardinality

Variable	Unique values	Encoding strategy
category	14	One-hot
gender	2	One-hot
state	51	—
job	497	Frequency encoding
merchant	693	Frequency encoding
city	906	Frequency / smoothing

Transaction amount

The log-transformed amount distribution differed visibly between genuine and fraudulent transactions, motivating the use of amount-based ratio.



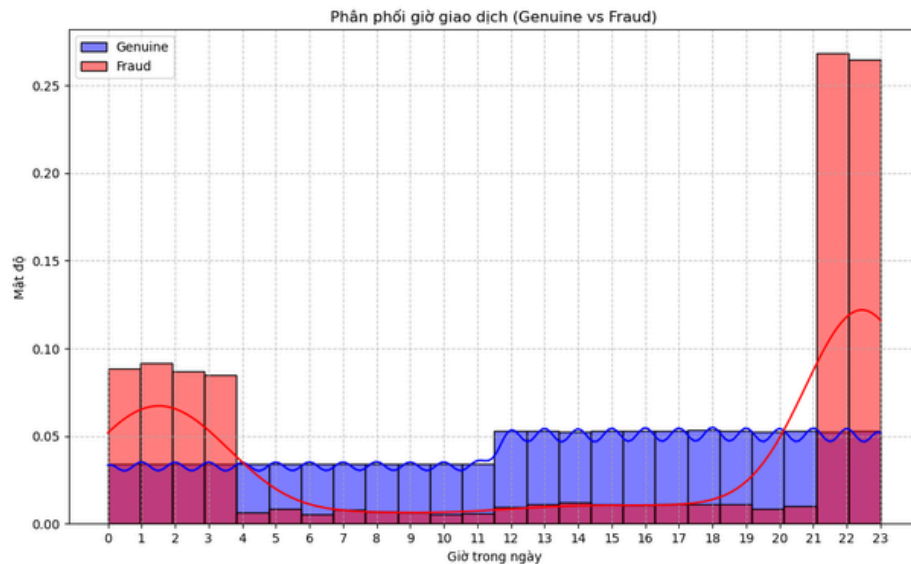
Amount-to-average ratio

Binning `amt_to_avg_ratio` revealed a strong, monotonic relationship with fraud incidence (Table 1) — rising from 0.29% of transactions at ratio ≤ 0.5 to 23.39% of transactions at ratio > 8 . A transaction disproportionately larger than a card's own historical average is therefore one of the strongest behavioural indicators in the dataset.

	transactions	fraud	fraud_rate
ratio_bin			
(-0.001, 0.5]	544741	1589	0.291698
(0.5, 1.0]	327285	204	0.062331
(1.0, 1.5]	195578	212	0.108397
(1.5, 2.0]	92202	196	0.212577
(2.0, 3.0]	82923	347	0.418460
(3.0, 4.0]	18449	423	2.292807
(4.0, 5.0]	9957	604	6.066084
(5.0, 6.0]	7146	500	6.996921
(6.0, 8.0]	5518	419	7.593331
(8.0, inf]	12876	3012	23.392358

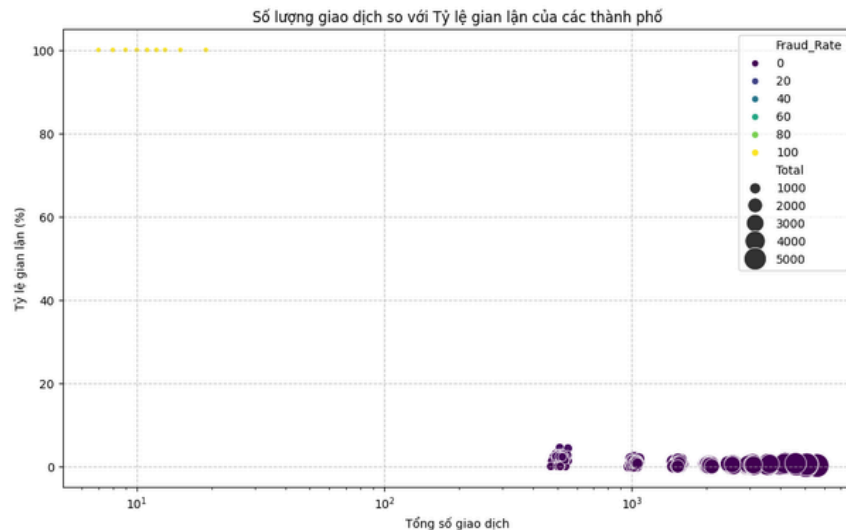
Hour of day

The distribution of transaction hour differed between genuine and fraudulent cases, motivating the cyclical sin/cos encoding used to represent the 24-hour periodicity for tree- and margin-based models

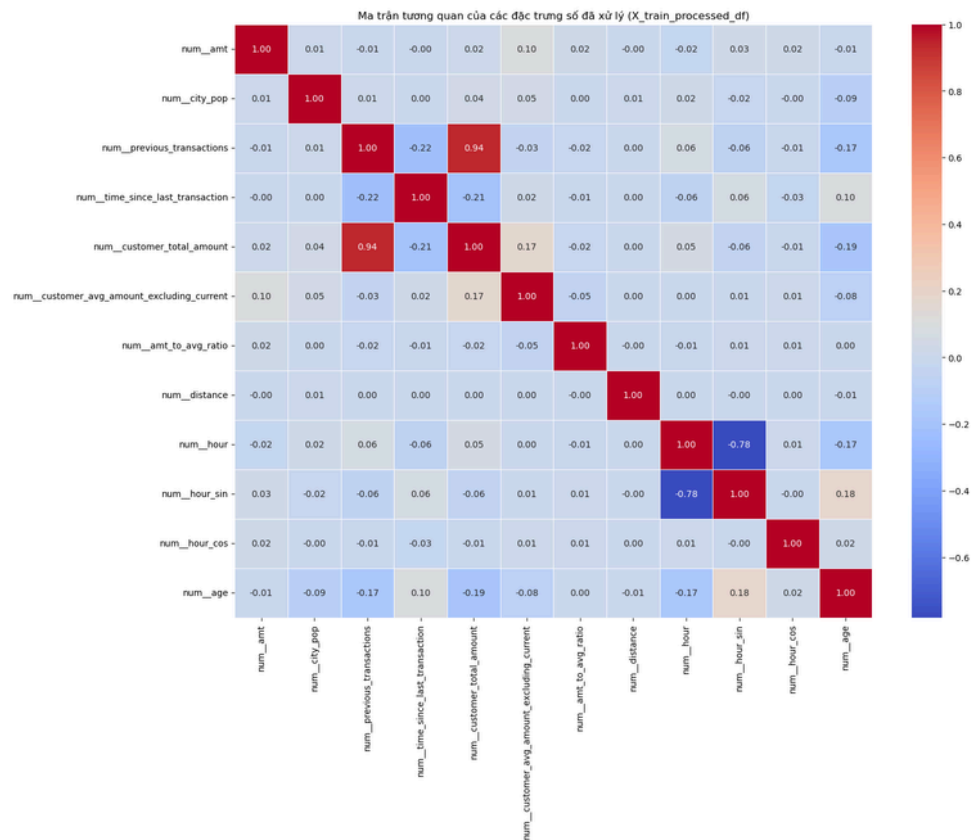


City-level fraud rate

A scatter analysis of raw per-city fraud rate against transaction volume showed that several low-volume cities exhibited a 100% raw fraud rate — an artefact of small sample size rather than genuine city risk — which motivated Bayesian smoothing of the exploratory city_fraud_encoded feature toward the overall fraud rate



Multicollinearity screening



FEATURE ENGINEERING

Feature Engineering Overview

- Goal: enrich raw transaction attributes with historical, contextual, and cyclical information
- 4 feature groups: **Behavioural, Interaction, Frequency encoding, Cyclic time**
- Key principle applied throughout: every feature uses only information available before the current transaction (no future leakage)

Behavioural Features

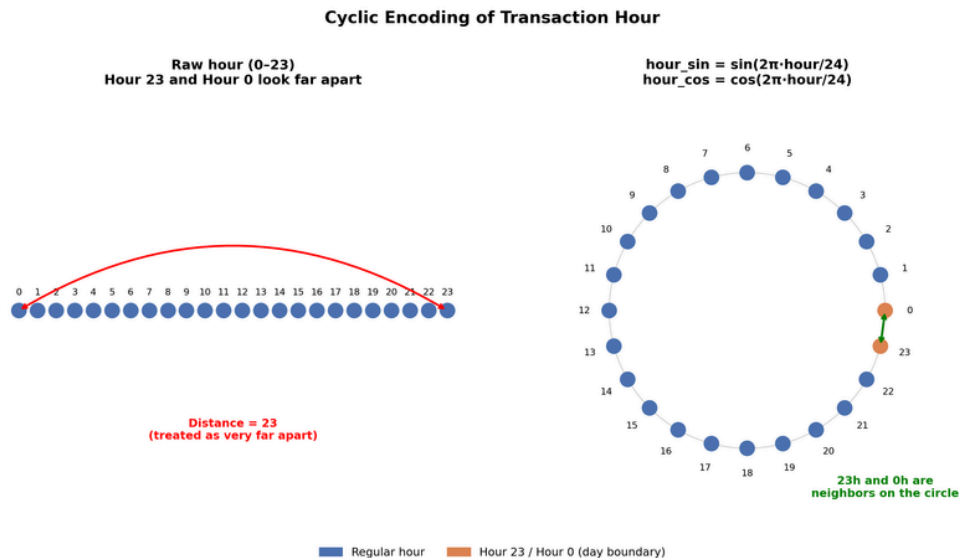
- `previous_transactions` — count of prior transactions per customer
 - `time_since_last_transaction` — seconds since last transaction
 - `customer_total_amount`, `customer_avg_amount_excluding_current` — cumulative & average historical spending, excluding current transaction
 - All computed causally (expanding, prior-only) — no future-information leakage
- captures customer-level spending history and preferences

Interaction Features

- `amt_to_avg_ratio` = current amount $\div$ customer's historical average → flags deviation from typical spending
- `distance` — Haversine distance between customer and merchant location (accounts for Earth's curvature)
- `hour` — extracted from transaction timestamp

Cyclic Time Features

- Problem: hour (0–23) implies false distance between hour 23 and hour 0
- Solution: sine–cosine encoding
- $\text{hour_sin} = \sin(2\pi \cdot \text{hour} / 24)$
- $\text{hour_cos} = \cos(2\pi \cdot \text{hour} / 24)$
- Places hours on a unit circle
→ adjacent hours stay close, including the midnight wrap-around



Feature Engineering Summary: Computed vs. Used

Feature	Computed?	Used in final model?
previous_transactions, time_since_last_transaction	✓	✓
customer_total_amount, customer_avg_amount_excluding_current	✓	✓
amt_to_avg_ratio, distance, hour, hour_sin/hour_cos	✓	✓
age	✓	✓
distance_x_hour	✓	✗
merchant_freq, job_freq	✓	✗
log_amount	✓	✗

DATA PREPARATION

Train / Validation / Test Split

- Time-based split (not random) → preserves chronological order, prevents future-info leakage
- fraudTrain.csv sorted by timestamp → split 80:20
- Train: 1,037,340 records (01/2019 – 03/2020)
- Validation: 259,335 records (03/2020 – 06/2020)
- No temporal overlap between train and validation (verified)
- Fraud rate: 0.58% (train) vs 0.59% (valid) → consistent across split
- fraudTest.csv (555,719 records) held out entirely — used only once, for final evaluation

Categorical Variable Analysis

Variable	Cardinality	Encoding
gender	2	(not used in final model)
category	14	One-hot
job	494	(not used in final model)
merchant	693	(not used in final model — replaced by behavioural/interaction features)
city	894	(not used in final model)

Only category is retained as a categorical input to the model — a simplification from earlier design iterations (frequency encoding for merchant/job was tested but ultimately not included in the final pipeline)

Imputation

- SimpleImputer included in the pipeline as a safeguard:
- Numerical features → median imputation
- Categorical features → most-frequent (mode) imputation
- Dataset confirmed to have zero missing values at the EDA stage → imputation currently has no visible effect on this dataset, but protects the pipeline against missing values in future/unseen data

Standardization

- StandardScaler applied to all numerical features after imputation
- Rescales each feature to mean = 0, standard deviation = 1
- Important given the wide value range across numerical features (e.g. amt, city_pop, distance) — prevents large-magnitude features from dominating distance- or gradient-based models (e.g. Logistic Regression)

One-Hot Encoding

- Applied only to category (14 values) via `OneHotEncoder(handle_unknown="ignore")`
-  Previous issue resolved: earlier notebook versions one-hot encoded `favorite_merchant` (high cardinality), producing hundreds of sparse columns — this has been removed from the final pipeline

ColumnTransformer & Pipeline

- All transformations (imputation, scaling, one-hot) combined into a single `ColumnTransformer`
- Fitted only on `train_df`; applied unchanged to `valid_df/test_df`
- Ensures a single, leakage-safe, reproducible preprocessing path across all three datasets

Processed Dataset

- Final feature matrix: 26 columns (previously 367)
 - 12 numerical features: amt, city_pop, previous_transactions, time_since_last_transaction, customer_total_amount, customer_avg_amount_excluding_current, amt_to_avg_ratio, distance, hour, hour_sin, hour_cos, age
 - 14 one-hot columns from category
- Much more compact, interpretable feature space than earlier iterations

MODEL DEVELOPMENT

Overview

- Three candidate models compared: Logistic Regression, XGBoost, Random Forest
- All trained on the same preprocessed feature matrix (367 columns) from Section 3.4
- Same train_df / valid_df split used for training and threshold tuning; test_df reserved for final evaluation only

Logistic Regression

- Baseline linear model — interpretable, fast to train
- Configuration: `solver='liblinear'`, `max_iter=1000`, `random_state = 42` (increased for convergence on a large dataset)
- Trained directly on the scaled + one-hot encoded feature matrix
- Evaluated on `valid_df` using classification report and confusion matrix
- Role in the study: interpretable baseline to benchmark tree-based models against

Random Forest

- `RandomForestClassifier` (`random_state=42`, `n_jobs=-1`) — default hyperparameters, no class weighting
- Strong performance out-of-the-box despite no explicit imbalance handling
- Evaluated at best threshold (0.21)

XGBoost

- Gradient-boosted tree ensemble — captures non-linear relationships and feature interactions
- Configuration: `n_estimators=100`, `max_depth=6`, `learning_rate=0.3`, `objective='binary:logistic'`
- `scale_pos_weight` set to the train class ratio (≈ 172) $\rightarrow$ directly compensates for the 0.58% fraud rate during training
- Classification threshold tuned on `valid_df` to maximise F2-score (`best_threshold_f2`)
- Feature importance extracted post-training to identify the top predictive features

Model Comparison (Validation Set)

Model	F1-score	F2-score
Logistic Regression	0.409	0.320
Random Forest	0.865	0.823
XGBoost	0.859	0.892

- XGBoost improves substantially on every fraud-class metric
- Largest gains: recall (0.38 $\rightarrow$ 0.94) and F2 (0.376 $\rightarrow$ 0.773)

Model Comparison Summary

- Logistic Regression → interpretable linear baseline
- XGBoost → currently the most developed model, with class-imbalance handling and F2-optimised threshold
- Random Forest → pending implementation
- Final comparison (once complete) to be based on F2-score, precision, recall on the held-out test set

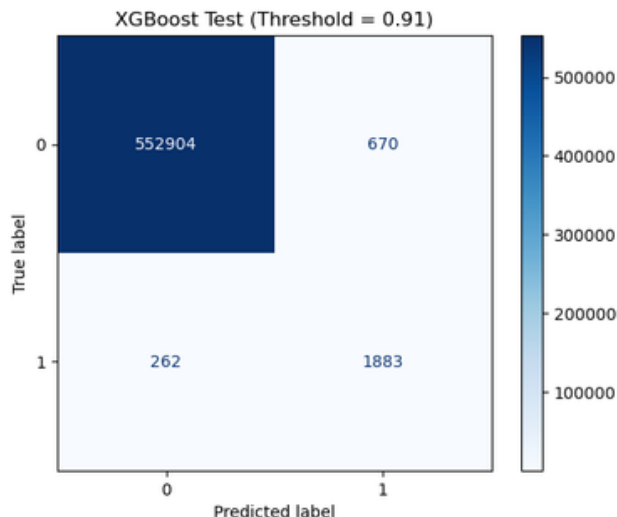
MODEL EVALUATION METRICS

XGBoost Performance (Validation vs Test Set, threshold = 0.91)

Metric	Validation	Test
Precision	0.81	0.74
Recall	0.91	0.88
F1-score	0.86	0.80
F2-score	0.89	0.85

- Threshold used: 0.91 (selected via F2 on validation, then applied unchanged to test)
- Some drop from validation → test (expected) — but performance remains strong and consistent, no evidence of severe overfitting to validation

Confusion Matrix Interpretation



- **True Positives:** 1,883 fraud cases correctly caught
 - **False Negatives:** 262 fraud cases missed
 - **False Positives:** 670 genuine transactions flagged
 - **True Negatives:** 552,904 correctly passed
- Model catches **~88% of fraud** on entirely unseen data, at the cost of flagging ~670 genuine transactions for review

Feature Importance (Top Predictors)

1. amt (0.169)
2. hour_cos (0.148)
3. category_gas_transport (0.115)
4. category_food_dining (0.067)
5. category_shopping_pos (0.054)
6. category_shopping_net (0.052)
7. category_grocery_net (0.048)
8. category_home (0.047)
9. category_grocery_pos (0.044)
10. category_entertainment (0.041)

- Transaction amount and time-of-day (cyclic hour) are the two strongest single predictors
 - Merchant category — particularly gas/fuel — remains a strong fraud signal, consistent with EDA
- ✓ No longer dominated by individual high-cardinality merchant dummies (resolved vs. earlier notebook version — favorite_merchant is no longer in the pipeline)

Bootstrap Stability Check (Test Set, 100 resamples)

Metric	Mean	Std Dev	Min	Max
F2-score	0.846	0.007	0.828	0.860
Precision	0.738	0.009	0.716	0.761
Recall	0.878	0.007	0.857	0.895

- Narrow spread (std < 0.01) → model performance is stable across resamples, not driven by a few favourable cases
- Bootstrap performed on the test set (fixed model, fixed threshold 0.91)
- Scope: measures sampling variability only — not retraining variability

Demo: Interactive Dashboard

